

Quantum Encoding Agents: A Natural Language Interface for Data Embedding Strategy Selection in Quantum Machine Learning

Ana Paula Appel
aappel@redhat.com
Red Hat, Inc.
USA

CCS Concepts

• **Computing methodologies** → **Quantum computing**; *Supervised learning*; *Natural language processing*; • **Security and privacy** → Access control.

Keywords

Quantum Machine Learning, Quantum Feature Maps, Data Encoding, AI Agents, NISQ Hardware, OpenShift, Natural Language Interface

1 Introduction

The practical adoption of Quantum Machine Learning faces a fundamental bottleneck that precedes model training: the problem of quantum data encoding. Before any quantum classifier, kernel method, or variational circuit can operate, classical data must be mapped into quantum states. This mapping—the encoding or feature map—is not a neutral preprocessing step. It defines the geometry of the state space, the structure of correlations captured between features, and the depth and qubit count of the resulting circuit. Different encodings impose radically different tradeoffs: amplitude encoding compresses n features into $\lceil \log_2 n \rceil$ qubits but requires a state preparation circuit of exponential depth; angle encoding produces shallow circuits with one qubit per feature but limited expressibility; IQP encoding [?] captures cross-feature correlations through diagonal unitaries and has strong theoretical guarantees for quantum kernels, but at greater circuit depth.

Despite the critical nature of this choice, the QML literature offers little guidance on encoding selection as a function of data characteristics. Practitioners encounter a combinatorial decision problem: seven or more encoding families, each with different qubit efficiency, circuit depth, hardware sensitivity, and affinity for different QML algorithms, must be matched against a dataset whose relevant properties—dimensionality, value type, distribution, sign—interact with the encoding in non-obvious ways. The seminal survey by Sammartino [?], reviewing 66 papers from 2017 to 2026, identifies this as the most underaddressed problem in applied QML, formalizes six encoding families, and derives a critical gate error rate threshold $p^* \approx 10^{-3}$ above which deep encodings become impractical on NISQ hardware.

This paper addresses the encoding selection problem through a different lens: rather than proposing a new encoding or

a new theoretical framework, I build a practical agent system that makes existing knowledge actionable. I make the following contributions:

- (1) **A hardware-aware encoding recommendation engine** that applies the p^* threshold from [?], accounting for gate error rate, circuit depth budget, qubit count, and chip connectivity topology.
- (2) **Seven implemented encoding strategies** in Qiskit, including dense-angle encoding [?] and IQP encoding [?], which were identified as underrepresented in applied tooling.
- (3) **A natural language explanation layer** that generates structured justifications in Portuguese and English, citing concrete metrics (number of qubits, circuit depth, Kernel-Target Alignment) derived from actual simulation results.
- (4) **A quantum kernel evaluation endpoint** that computes $K[i, j] = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$ for a dataset, returns KTA scores, and generates a heatmap visualization—enabling encoding quality assessment before any classifier training.
- (5) **A multi-agent deployment architecture** on Red Hat OpenShift with NVIDIA OpenShell security sandboxes, exposing three OpenClaw agents with distinct communicative personalities for expert and novice audiences.
- (6) **A Kubeflow Pipeline** for Red Hat OpenShift AI that orchestrates the full workflow—data analysis, encoding recommendation, comparison, kernel evaluation, and MLflow artifact logging—as a reproducible five-stage DAG.

The system is released as open source.¹

2 Related Work

2.1 Quantum Data Encoding Strategies

The theoretical landscape of quantum data encoding has been shaped by a sequence of foundational papers. Schuld and Killoran [?] established the connection between quantum feature maps and kernel methods, showing that quantum circuits define kernels on data through the inner product of encoded states in Hilbert space. This framing—encoding

¹API: <https://github.com/anapaulaappel/quantum-encoding-agents>; Agents: <https://github.com/anapaulaappel/openclaw-quantum-agents>. All code, policies, agent configurations, and the Kubeflow pipeline were developed and are maintained by the author.

as an implicit kernel—is the basis for all kernel-based QML methods.

Havlíček et al. [?] operationalized this idea with a concrete circuit construction: the ZZ feature map, which applies Hadamard layers followed by diagonal Pauli rotations encoding both linear (x_i^2) and cross-product ($x_i \cdot x_j$) terms. This circuit, equivalent to what we term IQP (Instantaneous Quantum Polynomial) encoding, was demonstrated on IBM quantum hardware and showed experimental advantage over classical SVMs on a constructed classification task.

Pérez-Salinas et al. [?] introduced data re-uploading, departing from the one-shot encoding paradigm. In their framework, classical data is inserted into the circuit multiple times across layers, interleaved with trainable parameters. Data re-uploading has become the de facto encoding for variational quantum classifiers and quantum neural networks.

LaRose and Coyle [?] conducted the first systematic comparative study of encoding strategies across multiple datasets and noise conditions, establishing that no single encoding dominates and that the optimal choice is dataset-dependent. Their work motivates the need for an automated selection mechanism—a gap this system addresses.

The most comprehensive treatment is Sammartino’s 2026 survey [?], which reviews 66 papers from 2017 to 2026, classifies encodings into six families, and derives practical selection guidelines. Crucially, Sammartino formalizes the critical gate error threshold: for hardware with gate error rate $p \geq p^* \approx 10^{-3}$, deep encodings (amplitude, IQP, custom feature map) accumulate noise faster than their expressibility advantage compensates, making shallow encodings (angle, dense angle, data re-uploading) preferable. Dense-angle encoding—which packs two features per qubit via $R_y(x_{2i}) \cdot R_z(x_{2i+1})$ —is identified as the most underused encoding family in applied QML despite its favorable depth-qubit tradeoff.

2.2 Encoding as an Implicit Kernel

A result fundamental to the theoretical grounding of this system is Schuld’s 2021 theorem [?]: every supervised quantum model that encodes data into a quantum state $|\phi(x)\rangle$ and measures an observable is mathematically equivalent to a kernel support vector machine with kernel:

$$K(x, x') = |\langle \phi(x) | \phi(x') \rangle|^2. \quad (1)$$

This equivalence has a decisive practical implication: *“the way that data is encoded into quantum states is the main ingredient that can potentially set quantum models apart from classical machine learning models”* [?]. Furthermore, kernel-based training—which is what QSVM performs directly—is provably at least as good as variational circuit training for the same encoding. The choice of encoding is therefore not a preprocessing decision; it is the core modeling decision, equivalent to choosing a kernel in classical SVM. This system makes this choice explicit and measurable—the Kernel-Target Alignment (KTA) score computed by the $\sqrt{1/\text{kernel}}$ endpoint is a direct, label-aware evaluation of the implicit kernel defined by any encoding on the actual data.

2.3 Trainability and Barren Plateaus

The trainability of quantum circuits is a central concern for encoding choices used in variational algorithms. McClean et al. [?] demonstrated that randomly initialized quantum circuits exhibit barren plateaus: exponentially vanishing gradients that make training infeasible beyond moderate qubit counts. Cerezo et al. [?] surveyed the landscape of variational quantum algorithms, establishing data re-uploading as the preferred input encoding for QNNs and VQCs due to its per-layer data injection that maintains gradient flow.

2.4 Expressibility, Entanglement, and the Encoding Tradeoff

Sim, Johnson and Aspuru-Guzik [?] define *expressibility* as the deviation of a parameterized quantum circuit’s output distribution from the Haar measure. Their key result: shallow circuits have low expressibility and are confined to structured submanifolds of the state space, which may be insufficient to separate classes. This is the complementary risk to the barren plateau: too-shallow encodings underfit in Hilbert space, while too-deep circuits become untrainable due to vanishing gradients [?]. Sammartino [?] adds the third axis: above $p^* \approx 10^{-3}$, the depth budget is set by hardware decoherence, not by expressibility requirements.

Larocca et al. [?] further establish that over-parameterized quantum neural networks undergo a phase transition from a barren plateau regime to a trainable regime as the number of parameters exceeds a critical threshold, directly linking encoding circuit depth to parameter count and trainability.

2.5 Quantum Advantage and Scope of NISQ-Era QML

Huang et al. [?] proved exponential quantum advantage in a specific learning task—predicting properties of quantum physical systems from quantum measurements—demonstrated on 40-qubit superconducting hardware. However, this advantage is task-specific: it applies when the data itself is quantum in origin, not when encoding classical tabular data into quantum circuits. Huang, Kueng and Preskill [?] establish complementary information-theoretic bounds showing that classical ML can match quantum ML on average over input distributions, but quantum advantage is possible worst-case.

For classical datasets on NISQ hardware—the regime this system targets—quantum advantage over classical methods has not been proven in general. Bowles, Ahmed and Schuld [?] show, in a large-scale study across 12 QML models and 160 datasets, that classical baselines frequently match or outperform quantum classifiers at current qubit scales, underscoring that encoding evaluation is not optional but essential. This system is therefore designed as a NISQ-era engineering tool for informed encoding selection, not as a claim of general quantum supremacy.

2.6 NISQ vs. Fault-Tolerant Quantum Computing

The `hardware_profile` constraints in this system encode NISQ-era realities that will shift significantly in the fault-tolerant regime. On current NISQ devices—characterized by gate error rates of 10^{-3} to 10^{-2} , qubit counts of 10–400, and no error correction—circuit depth is the primary engineering constraint. In the fault-tolerant era, with logical qubits suppressing error to arbitrarily low levels, this constraint disappears: amplitude encoding’s exponential qubit compression becomes viable, and the encoding choice is driven purely by expressibility and task alignment. This design is forward-compatible: passing `gate_error_rate: 0.0` returns recommendations based on expressibility alone; passing `gate_error_rate: 5e-3` (IBM Eagle) returns hardware-adjusted recommendations.

2.7 AI Agents for Scientific Tooling

The use of LLM-based agents as interfaces for scientific computing tools is an emerging paradigm. To my knowledge, this is the first system to deploy multi-personality LLM agents specifically for quantum encoding recommendation.

The OpenClaw agent framework [?] provides the infrastructure for agent deployment with persistent memory (via `MEMORY.md`), user calibration (via `BOOTSTRAP.md` and `USER.md`), and skill encapsulation.

2.8 Secure Agent Deployment

NVIDIA OpenShell [?] addresses agent security through out-of-process policy enforcement: each agent runs in an isolated sandbox governed by a YAML policy that constrains filesystem access and network endpoints at HTTP method granularity, enforced by Landlock LSM and seccomp. The policy engine validates rules using the Z3 SMT solver before application. In this deployment, each agent receives a distinct policy: the expert agent (Circuit) is restricted to the Qiskit API and LLM endpoints only; the mentor agent (Quanta) additionally has read-only access to arXiv and Qiskit documentation.

3 Problem Description

3.1 The QML Training Pipeline

To position the encoding selection problem, I first establish the full QML training loop in which it occurs. A supervised quantum machine learning pipeline consists of five stages executed iteratively:

- (1) **Data Encoding:** $U(x)|0\rangle^{\otimes n} \rightarrow |\phi(x)\rangle$ (*this paper*)
- (2) **Parameterized Ansatz:** $V(\theta)|\phi(x)\rangle \rightarrow |\psi(x, \theta)\rangle$
- (3) **Measurement:** $\langle\psi(x, \theta)|M|\psi(x, \theta)\rangle \rightarrow \hat{y}$
- (4) **Classical Loss:** $\mathcal{L}(\hat{y}, y) \rightarrow \mathbb{R}$
- (5) **Classical Optimizer:** $\theta \leftarrow \theta - \eta \nabla \mathcal{L}$ (then repeat)

The encoding layer (stage 1) is fixed for the duration of training—it is a hyperparameter of the model, not a trainable parameter. A poorly chosen encoding defines a kernel that cannot linearly separate the classes in Hilbert space, no

matter how many training iterations the optimizer runs. This system operates at stage 1, with KTA providing a measurement of whether the chosen encoding creates a Hilbert space geometry useful for the loss at stage 4.

For quantum kernel methods (QSVM), stages 2–5 are replaced by classical SVM training on the kernel matrix $K[i, j] = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$ —which is exactly what the `/v1/kernel` endpoint computes. In this case, the encoding is the *entire* model specification, reinforcing the centrality of stage 1.

3.2 The Encoding Selection Problem

Let $\mathcal{D} = \{x_i\}_{i=1}^N \subset \mathbb{R}^d$ be a classical dataset with N samples and d features. A quantum encoding is a parameterized map $\phi: \mathbb{R}^d \rightarrow \mathcal{H}$ from the feature space to a Hilbert space $\mathcal{H}$ of dimension 2^n , implemented as a quantum circuit $U(x)$ acting on n qubits such that $|\phi(x)\rangle = U(x)|0\rangle^{\otimes n}$.

The encoding selection problem is: given $\mathcal{D}$ and an optional QML task specification $\mathcal{T}$ (classification, kernel method, variational circuit, etc.) and hardware constraints $\mathcal{Hw}$ (gate error rate, qubit count, connectivity topology), select an encoding ϕ^* that maximizes downstream task performance subject to hardware feasibility.

This problem is ill-posed for two reasons. First, the downstream task performance depends on the full learning pipeline, not just the encoding; it cannot be evaluated without training a model. Second, the interaction between data characteristics and encoding properties is non-linear and high-dimensional.

3.3 Current Practice and Its Limitations

In practice, encoding selection is rarely principled. A survey of QML implementations in the literature reveals that the majority use angle encoding as a default, regardless of dataset structure. However, this choice ignores:

- **Dimensionality:** for $d > 10$ features, angle encoding requires d qubits. Dense-angle encoding achieves the same depth with $\lceil d/2 \rceil$ qubits.
- **Task alignment:** for QSVM, encodings without entanglement define kernels with limited expressibility. IQP and custom feature map encodings define richer kernels.
- **Hardware noise:** above $p^* \approx 10^{-3}$, amplitude encoding’s deep state preparation circuit accumulates noise faster than its qubit compression advantage is worth. On IBM Eagle hardware ($p \approx 5 \times 10^{-3}$), the effective circuit depth after error accumulation can reduce the signal-to-noise ratio below classification threshold.
- **Data type:** basis encoding, the natural choice for binary or categorical data, is systematically ignored when practitioners default to angle encoding for all data types.

3.4 The Accessibility Gap

Beyond the technical selection problem, there is an accessibility gap: the knowledge required to make principled encoding

choices—distributed across a dozen papers published between 2018 and 2026, requiring familiarity with quantum information theory, circuit complexity, and NISQ hardware characteristics—is not accessible to the ML practitioner approaching QML for the first time.

This gap manifests in two distinct user profiles. The **QML practitioner** (expert) needs dense, metric-first information: encoding name, circuit depth, qubit count, KTA score, and a reference to the relevant paper. The **ML practitioner approaching QML** (learner) needs progressive disclosure: physical intuition before mathematics, the Bloch sphere before Dirac notation, a concrete analogy before a formal definition. Current tools address neither profile. The system I present bridges this gap through specialized agents calibrated to each profile.

4 Quantum Encoding Agents: System Design

4.1 Architecture Overview

The system consists of three loosely coupled layers: (1) an OpenClaw agent gateway with LLM backend (Qwen2.5-Coder-14B via Ollama or Llama-3.2-3B via vLLM on OpenShift), (2) the llama-qiskit-agents FastAPI microservice exposing the quantum pipeline, and (3) the NVIDIA OpenShell security layer enforcing per-agent sandbox policies. The microservice is stateless and can be called directly via HTTP independently of the agent framework.

4.2 Data Profile Analysis

The first processing step is structured data profiling. Given input in any of three forms—a numpy array, a CSV file, or a natural language description—the system extracts a `DataProfile` dataclass with seven fields: `n_samples`, `n_features`, `is_binary`, `is_categorical`, `is_continuous`, `has_negative`, and `description`. For text descriptions, keyword detection in both Portuguese and English maps phrases to profile flags.

4.3 The Seven Encoding Strategies

The system implements seven encoding families as Qiskit `QuantumCircuit` objects. Table 1 summarizes their properties.

Table 1: Quantum encoding strategies implemented in the system.

Encoding	Qubits	Depth	Recommended for
Amplitude	$\lceil \log_2 n \rceil$	$O(2^n)$	Large vectors, qubit-constrained
Angle	d	1	$d \leq 4$, continuous
Dense Angle [?]]	$\lceil d/2 \rceil$	2	$5 \leq d \leq 12$, contin.
IQP [?]]	d	$\sim 3d$	$8 \leq d \leq 16$, kernels
Basis	d	$\leq d$	Binary / categorical
Data Re-upl. [?]]	d	$\sim 4d$	VQC, QNN
Custom FM	d	$\sim 3d$	QSVM, arbitrary kernels

Dense-angle encoding [?] achieves half the qubit count of standard angle encoding at depth 2 by packing two features per qubit:

$$U_{\text{DA}}(x) |0\rangle^{\otimes \lceil d/2 \rceil} = \bigotimes_{i=0}^{\lceil d/2 \rceil - 1} R_z(x_{2i+1}) R_y(x_{2i}) |0\rangle_i. \quad (2)$$

IQP encoding [?] is implemented without a qiskit-machine-learning dependency through manual decomposition of $R_{zz}(\theta) = CX \cdot R_z(\theta) \cdot CX$:

$$U_{\text{IQP}}(x) = H^{\otimes d} \cdot \prod_i R_z(x_i^2) \cdot \prod_{i < j} R_{zz}(x_i x_j) \cdot H^{\otimes d}. \quad (3)$$

Only adjacent pairs $(i, i+1)$ are connected, keeping circuit depth polynomial in d and compatible with linear and heavy-hex topologies.

4.4 Hardware-Aware Recommendation Policy

The recommendation pipeline has two stages: data-driven base selection and hardware-constrained refinement.

Base selection applies a priority decision tree over the `DataProfile`: (1) binary, $d \leq 16 \rightarrow$ `basis`; (2) continuous, $d \leq 4 \rightarrow$ `angle`; (3) continuous, $4 < d \leq 12$, no negatives $\rightarrow$ `dense_angle`; (4) single sample, $d \geq 4 \rightarrow$ `amplitude`; (5) continuous, $8 < d \leq 16 \rightarrow$ `iqp`; (6) continuous, $d > 16 \rightarrow$ `custom_feature_map`; (7) default $\rightarrow$ `angle`.

Task refinement overrides based on the QML task: kernel methods and QSVM elevate to `custom_feature_map`; variational methods prefer `data_reuploading`; binary classification with binary data uses `basis`.

Hardware refinement applies the p^* threshold from Sammartino [?]. For any `HardwareProfile` with `gate_error_rate` $\geq 10^{-3}$, deep encodings (`amplitude`, `custom_feature_map`, `iqp`) are replaced by `dense_angle` ($d > 4$) or `angle` ($d \leq 4$). Additionally, `connectivity`: heavy-hex or linear triggers a SWAP overhead warning for `custom_feature_map`, and `max_depth_budget` triggers a depth feasibility warning.

4.5 Natural Language Explanation Generation

The explanation layer generates a four-paragraph structured narrative in the detected language (Portuguese or English), each paragraph citing concrete values from the `DataProfile` and `SimulationResult`: (1) data description, (2) encoding key features with geometric intuition, (3) circuit metrics with NISQ compatibility assessment, and (4) comparative analysis of alternatives with concrete depth and qubit differences. Language detection uses vocabulary overlap between the input text and Portuguese/English marker sets.

4.6 Quantum Kernel Evaluation

For datasets where classification labels are available, the `QuantumKernel` computes the quantum kernel matrix $K \in [0, 1]^{N \times N}$:

$$K[i, j] = |\langle \phi(x_i) | \phi(x_j) \rangle|^2. \quad (4)$$

Statevectors are computed using the `StatevectorSimulator` (exact, without measurement shots) and inner products are computed classically. The Kernel-Target Alignment is computed as:

$$\text{KTA}(K, y) = \frac{\langle K, yy^\top \rangle_F}{\|K\|_F \cdot \|yy^\top\|_F}, \quad (5)$$

where $y \in \{-1, +1\}^N$ is the label vector. KTA provides a label-aware measure of how well the encoding separates classes in Hilbert space, independently of any trained classifier. The result is returned as a JSON matrix, a heatmap visualization (viridis colormap, per-class tick coloring), and a natural language caption.

In a preliminary evaluation on a synthetic two-class dataset ($N = 4, d = 2$), angle encoding achieved $\text{KTA} = 0.701$ while custom feature map achieved $\text{KTA} = 0.114$, demonstrating that richer encodings do not necessarily produce better-aligned kernels and that empirical evaluation via KTA is essential before committing to a training pipeline.

4.7 Bloch Sphere Visualization

For encodings with $n \leq 6$ qubits, the system optionally captures the statevector before measurement and renders the Bloch sphere representation of each qubit’s marginal state using `plot_bloch_multivector`. The resulting PNG (base64-encoded) accompanies the recommendation when `include_bloch: true` is set in the request, providing geometric intuition for what the encoding does to each data point.

4.8 Agent Personalities and the Role of SOUL.md

Three OpenClaw agents expose the recommendation system through distinct communicative registers. **QiskitAgent** (`quantum_encoding`) is the general-purpose agent. **Circuit** (`qiskit_expert`) is calibrated for QML practitioners: its `SOUL.md` instructs it to presuppose familiarity with key concepts, respond in three dense blocks (encoding name, metrics, rationale), never explain superposition, and present trade-offs as two objective sides without choosing. **Quanta** (`qiskit_mentor`) is calibrated for learners: its `SOUL.md` prohibits the Schrödinger’s cat analogy (which lies) and mandates physical intuition before mathematics—the Bloch sphere before Dirac notation. Both agents perform a calibration ritual on first session (`BOOTSTRAP.md`) to populate `USER.md` with a persistent user profile.

4.9 Skill Architecture

Two shared skills encapsulate reusable behavior across all agents. `$qiskit-api` documents the `/v1/recommend/explain` endpoint as a `SKILL.md` file, eliminating documentation duplication and ensuring consistent behavior when the API evolves. `$circuit-review` provides a four-step protocol for reviewing an existing circuit pasted by the user: identify the encoding, extract metrics, call `/v1/compare` for benchmark, and issue one of three verdicts (keep / optimize / replace) with a technical rationale.

4.10 Security Architecture (OpenShell)

Each agent runs inside a dedicated NVIDIA OpenShell sandbox governed by a YAML policy validated by the Z3 SMT solver. The policies enforce least-privilege at HTTP method granularity: `Circuit` (expert) has access limited to `qiskit-api:8080` (read-write) and the LLM endpoint (POST only); `Quanta` (mentor) additionally has read-only access to `arxiv.org` and `docs.quantum.ibm.com`; `QiskitAgent` (general) includes write access to `/tmp` for CSV uploads. Binaries are specified as objects with path keys, enabling per-binary policy enforcement through Landlock LSM.

4.11 Kubeflow Pipeline for RHOAI

The full workflow is codified as a Kubeflow Pipelines v2 DAG deployable on Red Hat OpenShift AI [?]: (1) `analyze_data` via POST `/v1/analyze`, (2) `recommend_encoding` via POST `/v1/recommend/exp`, (3) `compare_encodings` via POST `/v1/compare`, (4) `compute_kernel` via POST `/v1/kernel` (when labels are provided), and (5) `log_to_mlflow` [?] (parameters, metrics, heatmap, Bloch sphere visualization, generated Qiskit code). The pipeline accepts hardware profile parameters that propagate to the recommendation engine, enabling reproducible NISQ-aware experiments.

5 Discussion

5.1 The Role of KTA in Practice

My preliminary results suggest that KTA is a more reliable pre-training signal than circuit expressibility alone. Angle encoding ($\text{KTA} = 0.701$) outperformed the custom feature map ($\text{KTA} = 0.114$) on the test dataset despite the latter’s greater theoretical richness. This confirms LaRose and Coyle’s finding [?] that no encoding universally dominates, and motivates the design approach: rather than recommending based on theoretical properties alone, I compute KTA on the actual data and present the score alongside the recommendation.

5.2 Limits of Text-Based Encoding Selection

The natural language input path—inferring data profile from a description—has inherent limitations. Keyword detection can misclassify ambiguous descriptions. For production use, numerical data or CSV upload should be preferred; the text path is primarily useful for initial exploration and for the conversational interface.

5.3 Agent Personality as an Engineering Artifact

The `SOUL.md`-based personality specification is a form of behavioral engineering that sits between prompt engineering and agent architecture. Unlike a system prompt, `SOUL.md` is persistent (injected every session), versioned in git, and testable: a response from `Circuit` that begins with “Great question!” violates the soul specification and can be detected in evaluation. This framing—soul as contract—enables more

systematic quality assurance of agent behavior than ad-hoc prompting.

5.4 OpenShell Policy Design as Security-by-Data-Profile

The assignment of different network permissions to different agents based on their communicative role is a novel security design pattern: **policy as a function of agent persona**. Quanta requires external internet access (arXiv, Qiskit docs) because its role is pedagogical and citation-heavy. Circuit does not, because experts are expected to know the literature. This reflects the insight that the minimum necessary privilege of an AI agent is determined not just by its technical function but by its communicative purpose.

6 Conclusion

I presented Quantum Encoding Agents, a system that addresses the encoding selection problem in Quantum Machine Learning through a combination of hardware-aware recommendation logic, natural language generation, quantum kernel evaluation, and specialized AI agents calibrated to different

user expertise levels. The system is grounded in the theoretical framework of Sammartino [?] and Havlíček et al. [?], implements seven encoding families including the underused dense-angle encoding, and provides the first open-source implementation of KTA-based encoding quality assessment via a REST API.

The multi-agent architecture demonstrates that AI agents deployed for scientific tools can and should have distinct communicative personalities calibrated to their user populations—not as a cosmetic feature, but as a substantive design choice that determines what information to surface, in what order, and with what degree of assumed prior knowledge. The security architecture extends this idea: agent policy is a function of agent role, not just of agent function.

Future work includes: (i) integration with IBM Quantum real hardware via `QiskitRuntimeService` for empirical NISQ validation; (ii) the Q-Tucker shallow state preparation [?] as an alternative amplitude encoding with $O(n)$ depth; (iii) the Quantum Spectral Model [?] as an adaptive encoding whose frequency spectrum is conditioned on the input; and (iv) a KTA-guided encoding search that evaluates multiple candidates and returns the one with the highest alignment score.